CHAPTER THREE

SYSTEM METHODOLOGY AND DESIGN

3.1 Introduction

This chapter presents the methodology adopted in designing and developing the Abia State University (ABSU) Staff Database Management Portal. It provides a detailed exposition of the software development lifecycle, system analysis, functional and non-functional requirements, system architecture, database design, and the technical toolchain utilized during implementation. The objective is to provide a structured, replicable technical blueprint explaining how the web portal was conceived, architected, developed, and verified.

3.2 Software Development Methodology

The development of this project followed the Agile Development Methodology (Iterative and Prototyping Model). Staff records management involves diverse stakeholder roles (Super Administrators, Administrators, and Academic/Non-Academic Staff). An iterative approach allowed continuous verification, requirement refinement, and agile feature delivery.

3.2.1 Justification for Agile / Prototyping

• Iterative Feedback: Stakeholder requirements evolved during development—such as adding dual Super Admin rights, live multi-format report exports (PDF and Excel), and client-side photograph compression.

• Speed & Responsiveness: State transitions (e.g. from 'Submitted' to 'Approved' or 'Rejected') were incrementally implemented, tested, and validated without discarding previous progress.

• Risk Reduction: Continuous integration and unit compilation in a modern sandbox environment ensured that potential dependency conflicts or Firebase permission errors were caught and resolved early.

3.3 System Analysis & Requirements Specification

3.3.1 Functional Requirements

• Authentication & Access Control: Secure email/password account creation and sign-in via Firebase Authentication, accompanied by automated password recovery mechanisms.

• Role-Based Access Control (RBAC): Hierarchical privileges separating Super Administrators (sole authority to grant or revoke administrative roles), Administrators (verifying submissions, manual staff entry, CSV bulk uploads, deletions, announcements), and Staff Users (self-service profile entry and updates).

• Staff Profile Management: Capture of personal, employment, and biometric metadata: Full Name, Staff ID, Department, Faculty, Phone, DOB, Height, Blood Group, Address, Staff Type (Academic/Non-Academic), Photo, and CV.

• Verification & Approval Workflow: Submissions by regular staff are placed in 'Submitted' state until an Administrator explicitly reviews and clicks 'Accept' (Approved) or 'Reject'.

• Querying, Search & Analytics: Real-time substring search filtering across Name, Staff ID, Department, and Faculty, paired with graphical demographic metric charts.

• Reporting & Document Export: Instant client-side generation and download of formatted Adobe PDF directories and Microsoft Excel (.xlsx) workbooks.

• Campus Communication: Real-time staff message board and official university bulletins dispatch with audio support.

3.3.2 Non-Functional Requirements

• Performance & Responsiveness: Sub-second data filtering and instant live record updates through Firebase WebSocket subscriptions.

- **Security & Privacy: Encrypted password storage, JSON Web Token (JWT) session security, and client-side sanitization.**

- **Usability & Mobile Adaptation: Fully responsive glassmorphism aesthetic built on Tailwind CSS, supporting smartphones, tablets, and desktops.**

- **Data Integrity: Deterministic user document indexing ensuring staff updating their details update existing records without creating duplicate entries.**

3.4 System Architecture

The application adopts a modern Cloud-Native, Single Page Application (SPA) / Backend-as-a-Service (BaaS) architecture comprising two decoupled layers:

- **Presentation Layer (Frontend): Engineered with React 18, TypeScript, and Vite. Implements reactive UI states, dynamic routing, Lucide vector icons, and client-side processing engines (HTML5 Canvas for photo compression, jsPDF for PDF generation, SheetJS for Excel spreadsheets).**

- **Cloud Services Layer (Backend): Google Firebase Authentication manages cryptographic user tokens and password resets. Google Cloud Firestore provides high-performance, real-time distributed NoSQL document storage with live listeners (onSnapshot).**

3.5 Database Design and Data Modeling

The system utilizes a NoSQL schema optimized for read-heavy operations, sub-second queries, and flexible attributes across the following core collections:

Table 3.1: Staff Collection Schema (/staff)

Field	Data Type	Description
id	String	Unique auto-generated document identifier
name	String	Full legal name of staff member
employeeId	String	Official university employee/staff ID number
email	String	Registered account email address
faculty	String	Academic faculty / university division
department	String	Primary department assignment
phone	String	Active telephone contact number
dob	String (Date)	Date of birth
height	String	Physical height metric (cm)
bloodGroup	String	ABO blood group classification (A+, B+, O+, etc.)
address	String	Residential contact address
staffType	String	Classification: 'Academic' or 'Non-Academic'
photoUrl	String (Base64)	Compressed JPEG image rendered via HTML5 canvas
cvUrl	String (DataURI)	Curriculum vitae document attachment
role	String	Access tier: 'admin' or 'user'
status	String	Workflow status: 'Submitted', 'Approved', or 'Rejected'
createdAt	Timestamp	Server-generated submission timestamp

Table 3.2: Auxiliary System Collections

Collection	Key Attributes	Functional Purpose
users	uid, name, email, phone, role, createdAt	Stores operator credentials & role configurations
news	id, title, content, createdAt	Stores official dispatches, bulletins, and news announcements
messages	id, text, senderName, senderEmail, createdAt	Stores intra-institutional collaborative chat correspondence

3.6 System Workflow & Process Specifications

3.6.1 Staff Record Verification Lifecycle

When a staff member registers or updates their credentials, the record is flagged with a 'Submitted' status. Administrators review the entry within their dashboard, inspecting photographic and departmental data. The administrator can click 'Accept' (setting status to 'Approved') or 'Reject' (setting status to 'Rejected'). Real-time Firestore snapshot listeners immediately reflect this state change on the user's interface.

3.6.2 Role Escalation & Super Admin Governance

To prevent unauthorized privilege elevation, role modifications are guarded by a deterministic identity verification protocol. The system evaluates whether the authenticated user's email matches designated Super Administrator emails (ogungbadekehinde19@gmail.com or brossj50@gmail.com). Only upon verified identity match are 'Make Admin' or 'Remove Admin' controls enabled.

3.7 Implementation Toolchain and Technologies

Technology / Library	Category	Role in Project Implementation
React 18 & TypeScript	Frontend Framework	Type-safe, component-driven reactive single-page architecture
Vite	Build Tool	Rapid build bundling, development environment, and asset packaging
Tailwind CSS	Styling Engine	Glassmorphic responsive interface layout and design system
Motion (motion/react)	Animation	Fluid UI transitions, modal displays, and interactive animations
Lucide React	Iconography	Scalable vector interface icons
Recharts	Data Visualization	Demographic, status, and departmental analytical charts
jsPDF & jsPDF-AutoTable	Document Export	Client-side tabular PDF directory compilation and download
SheetJS (xlsx)	Spreadsheet Engine	Client-side generation and formatting of Excel (.xlsx) workbooks
Firebase Authentication	Identity & Security	Encrypted authentication tokens, session state, and password recovery
Google Cloud Firestore	Database (BaaS)	Distributed NoSQL cloud database providing live data sync
HTML5 Canvas API	Client Processing	In-browser downscaling and compression of staff photographs

3.8 Chapter Summary

This chapter presented the comprehensive methodology, architectural specifications, and database schema governing the ABSU Staff Database Management Portal. By integrating an Agile prototyping methodology with modern web technologies (React, TypeScript, Tailwind CSS) and Google Cloud Firestore services, the system delivers real-time synchronization, multi-tiered governance, client-side report exports, and efficient client-side asset optimization.